

TA-MPQ: STRUCTURED EXACT-BUDGET MIXED PRECISION FOR TASK-AWARE LLM QUANTIZATION

Anonymous Authors¹

ABSTRACT

Deploying large language models under strict memory budgets requires more than applying a uniform low-bit recipe to every layer. This paper presents TA-MPQ, a task-aware mixed-precision quantization pipeline that allocates INT2/INT4/INT8 precision under the same raw weight footprint as a uniform INT4 model. The method profiles group-level task sensitivity, builds a deterministic greedy path of unique budget-feasible policies, and selects coarse and refined candidates from real path snapshots rather than requested percentage targets. This draft follows the required paper format and reserves the Results section for the final quantitative tables, plots, and ablations. The current contribution is therefore the problem formulation, algorithmic design, implementation-grounded protocol, and explicitly stated limitations.

1 INTRODUCTION

Large language model deployment is often constrained by memory footprint and bandwidth rather than by model quality alone. Post-training quantization is a practical compression tool because it reduces model size without full retraining; SmoothQuant states the basic deployment motivation directly: “Quantization can reduce memory and accelerate inference” (Xiao et al., 2023). For local inference, edge deployment, and multi-model orchestration, however, the central question is not only whether a model can be compressed. The stronger question is how to allocate precision when the deployment budget is fixed in advance.

Uniform quantization is simple and reproducible, but it is also task-agnostic. A uniform INT4 policy assigns the same precision to every quantized linear component even though transformer components can differ in their importance for a target workload. The core intuition behind TA-MPQ is to treat precision as a scarce resource: high-sensitivity groups should receive more bits, low-sensitivity groups can sometimes pay for those promotions with fewer bits, and the remaining groups should stay at INT4.

Prior work supports the premise that not all weights or layers need equal treatment. AWQ argues that “weights are not equally important” and shows that protecting a small salient subset can reduce quantization error (Lin et al., 2024). You

Had One Job argues that task-aware quantization matters because common post-training pipelines may “spend bits on layers” that are less relevant to the target task (LeVi et al., 2026). These observations motivate task-aware mixed precision, but they do not by themselves specify how to construct a reproducible sequence of candidate policies that all satisfy the same deployment-sized budget.

The main technical challenge is that mixed-precision search is discrete and budget-coupled. Promoting group g from INT4 to INT8 costs $4n_g$ extra bits, where n_g is the group’s parameter count. Under a fixed uniform-INT4 raw footprint, that promotion must be offset by demoting other parameter mass from INT4 to INT2. This coupling makes free-form search unstable: a candidate that looks plausible before budget repair may be substantially rewritten after repair.

TA-MPQ addresses this issue by replacing unconstrained candidate generation with a deterministic exact-budget frontier. The algorithm starts at a uniform INT4 anchor, ranks groups by measured task sensitivity per true bit cost, greedily promotes high-value groups to INT8, and uses low-risk INT2 demotions to restore budget feasibility. Coarse candidates are evenly spaced snapshots along the realized greedy path of unique budget-feasible policies; refined candidates are local snapshots around the best coarse region.

This draft makes three contributions. First, it formulates task-aware mixed precision as exact raw-bit budget allocation rather than as a fixed-ratio recipe. Second, it describes a deterministic frontier construction and coarse-to-fine candidate selection procedure grounded in the current system. Third, it defines the experimental protocol and threat model that the final Results section must fill in.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

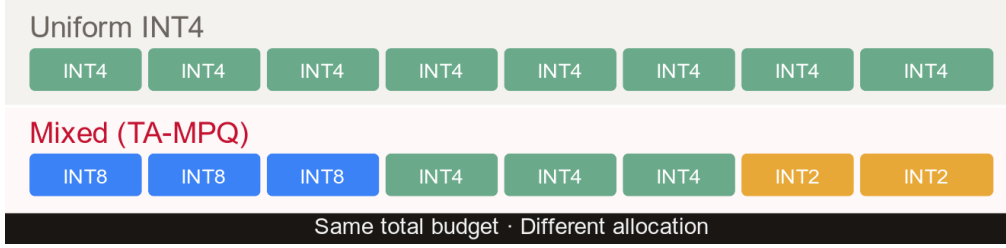


Figure 1. Budget-preserving precision reallocation. Uniform INT4 assigns the same precision everywhere, while TA-MPQ promotes selected high-sensitivity groups to INT8 and offsets the extra raw-bit cost with selected INT2 demotions.

2 RELATED WORK

Post-training quantization for LLMs. Post-training quantization compresses a pretrained model without retraining the full model. LLM.int8() showed that transformer inference can be made practical at 8-bit precision by treating outlier dimensions carefully (Dettmers et al., 2022). GPTQ frames large transformer compression as one-shot weight quantization and reports reducing weights to “3 or 4 bits per weight” with limited degradation (Frantar et al., 2023). SmoothQuant shifts quantization difficulty from activations to weights through an equivalent smoothing transformation (Xiao et al., 2023). These methods establish that low-bit post-training quantization can preserve useful behavior, but they mostly answer how to quantize under a chosen recipe rather than how to choose task-specific bit-width assignments under a strict footprint.

Activation-aware and task-aware quantization. Activation-aware methods use calibration signals to decide which values are important to preserve. AWQ protects salient weights using activation observations and explicitly states that “protecting 1% of salient weights” can reduce quantization error (Lin et al., 2024). You Had One Job extends this direction toward per-task mixed precision by using task-conditioned signals from hidden representations and sensitivity tests (LeVi et al., 2026). TA-MPQ uses the same broad principle as a profiler but differs in the search objective: every candidate must remain at or below the raw footprint of uniform INT4.

Mixed precision as budgeted allocation. Mixed-precision quantization treats bit-widths as allocation decisions rather than as a model-wide setting. This is a natural fit for transformers because attention projections, MLP projections, and output heads are not equally important for every workload. The difficulty is that the search space is combinatorial and the budget is discrete. TA-MPQ deliberately narrows this search to an ordered frontier so that each evaluated policy is feasible, interpretable, and reproducible.

Evaluation tasks for reasoning and coding. The final evaluation suite is intended to cover both recognition-style and generation-style tasks. HumanEval measures “functional correctness for synthesizing programs from docstrings” (Chen et al., 2021). EvalPlus augments HumanEval with stronger tests for generated code (Liu et al., 2023). BigCodeBench targets code generation with “diverse function calls as tools” and complex instructions (Zhuo et al., 2025). MATH introduces “12,500 challenging competition mathematics problems” and is useful for reasoning-heavy evaluation (Hendrycks et al., 2021). The Results section is reserved for the final numbers, but these benchmarks define the planned evidence base.

3 PROBLEM SETTING

Let a pretrained transformer contain a set of quantizable groups

$$G = \{g_1, g_2, \dots, g_m\}.$$

Each group g has parameter count n_g and receives a bit-width $b_g \in \{2, 4, 8\}$. In the current system, the grouping rule is per-block component assignment: each quantization group corresponds to one concrete linear module inside a transformer block or to a small number of special linear modules outside the repeated blocks. Examples include attention projections, MLP projections, and the language-model head.

The raw weight cost of a policy p is

$$B(p) = \sum_{g \in G} n_g b_g.$$

The target budget is the cost of uniform INT4:

$$B_{\text{INT4}} = \sum_{g \in G} 4n_g.$$

The feasible set is therefore

$$\mathcal{P} = \{p : B(p) \leq B_{\text{INT4}}\}.$$

This raw-bit accounting is the feasibility test for every generated policy. A policy may be under budget, but it is not

accepted if its quantized weight cost exceeds the uniform-INT4 target.

The optimization goal is to find a policy in $\mathcal{P}$ that performs well on a target task. This is not equivalent to maximizing the number of INT8 groups because group sizes vary. A small group can be promoted cheaply, while a large group can consume much of the promotion budget. Likewise, a small INT2 demotion may not offset an expensive INT8 promotion. The method must therefore reason over true parameter mass, not only group count.

4 METHOD

4.1 Overview

TA-MPQ converts task-aware mixed-precision search into a one-dimensional frontier over real budget-feasible policies. The pipeline has five stages. First, it enumerates quantizable linear groups and records their true parameter counts. Second, it estimates task sensitivity for each group. Third, it constructs a greedy path from uniform INT4 toward increasingly mixed policies. Fourth, it selects coarse and refined candidate snapshots from that path. Fifth, it quantizes and evaluates the selected policies as real artifacts.

4.2 Group Registry

The atomic search unit is a quantization group, not an entire transformer block. Under per-block component grouping, every group records a block index when applicable, a component type, a parameter count, and whether it may be quantized. This design keeps the search interpretable: a selected policy can be reported as assignments over components such as attention projections and MLP projections, rather than as an opaque vector of bits.

This granularity is a compromise. Whole-block assignment would be easier to serve and explain, but it is too coarse to distinguish attention and MLP submodules. Tensor-level assignment would be more flexible but would increase the search space and make policy interpretation harder. Per-block component grouping keeps the search space small enough for frontier construction while preserving meaningful transformer structure.

4.3 Task Sensitivity Profiling

The task sensitivity profiler is TAQ-KL-lite. It loads the model in BF16, prepares calibration prompts from the target task, records baseline next-token logits, and estimates how much a group-level perturbation changes the output distribution. For each group and candidate bit-width, the profiler injects quantization-shaped noise at that group’s linear module and computes KL divergence between the baseline and perturbed next-token distributions.

The profiler exports three quantities used by the frontier builder. The first is the estimated perturbation risk at INT4. The second is the nonnegative drop in risk from moving a group from INT4 to INT8. The third is the nonnegative increase in risk from moving a group from INT4 to INT2. The frontier builder uses the improvement from INT4 to INT8 as promotion value and the degradation from INT4 to INT2 as demotion cost.

4.4 Greedy Exact-Budget Path

The greedy path starts with every quantizable group at INT4. Candidate promotions are sorted by promotion value divided by the true extra bit cost $4n_g$. This makes promotion priority size-aware: a group must justify its precision increase relative to the number of additional bits it consumes.

When a trial promotion would exceed the target budget, the method searches for compensating demotions. It considers groups still at INT4, excludes the group being promoted, filters out groups with minimum-bit constraints, and ranks demotion candidates by demotion cost per saved bit. It then runs a local beam subset-cover procedure: each beam state tracks saved bits, accumulated demotion cost, and chosen groups; states with remaining shortfall are ranked behind states that cover the deficit; covered states are ranked by demotion cost plus an overshoot penalty. If the repaired trial policy is feasible, it becomes the next path state.

This path is intentionally monotone in construction but not monotone in expected accuracy. Moving along the path generally means adding more INT8 mass and the required INT2 compensation. Accuracy can rise or fall because each step changes multiple groups. The path is useful because it is real, deterministic, deduplicated by policy identity, and budget-feasible at every accepted step.

4.5 Coarse-to-Fine Selection

Coarse candidates are selected from the realized path itself. The selector deduplicates the path, removes the uniform INT4 anchor, removes the greedy max-INT8 endpoint, and places evenly spaced target points over the realized INT8 parameter-mass fraction of the remaining path. Each target is snapped to the nearest unused policy. Ties are resolved by lower realized INT2 mass, earlier path index, and finally policy identity.

Refinement is local on the same greedy path. The selector first chooses the best coarse policy after applying the configured accuracy tie band and tie-break rules. Optionally, it may include a second seed if that seed is within the allowed correct-answer gap, sufficiently separated in realized INT8 mass, and distinct by policy identity. Around each seed path index, the method builds a local window and selects evenly spaced unused policies.

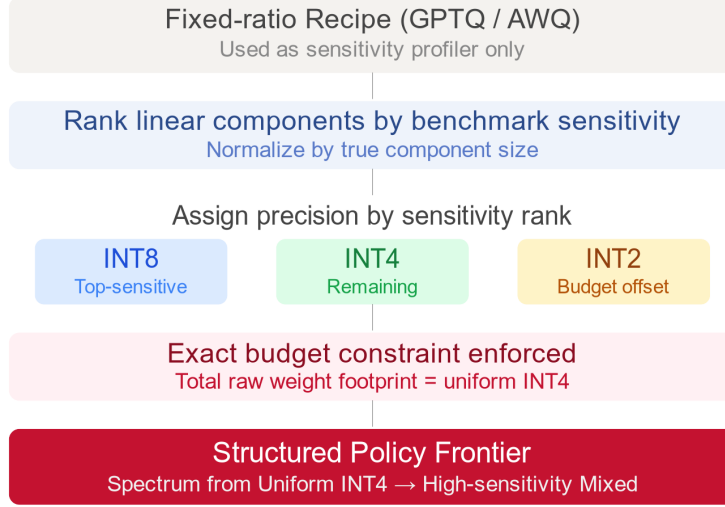


Figure 2. High-level TA-MPQ pipeline. Fixed-ratio quantization recipes are used only to obtain task sensitivity signals; the search then ranks groups by sensitivity per true bit cost, assigns INT8, INT4, and INT2 under an exact raw-footprint constraint, and produces a structured policy frontier.

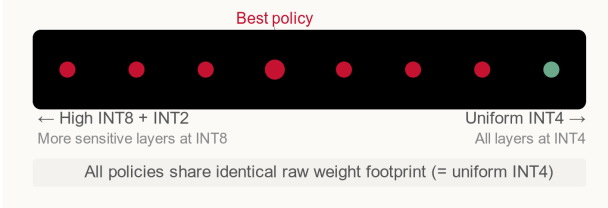


Figure 3. Coarse selection over the realized policy frontier. The search divides the frontier into sectors over realized INT8 parameter-mass fraction and evaluates real path snapshots rather than constructing candidates from requested percentages.

5 EXPERIMENTAL SETUP

Model and grouping. The active model is Qwen3.5-9B. The active bit space is $\{\text{INT2}, \text{INT4}, \text{INT8}\}$. The active group granularity is per-block component grouping. The budget is raw weight footprint less than or equal to uniform INT4. The language-model head is kept at least at INT4 as a minimum-bit constraint.

Search splits. The staged search has three evaluation phases. The coarse phase evaluates eight interior path snapshots on a shared question set. The refine phase evaluates four local path snapshots around the winning coarse region on a larger and disjoint question set. The final phase evaluates the selected winner on the held-out split. The uniform INT4 anchor is treated as a baseline rather than as a coarse candidate, and the greedy max-INT8 endpoint is diagnostic rather than part of coarse search.

Table 1. Algorithmic objects used by TA-MPQ. The table describes the paper-level algorithmic object rather than experiment entry-points.

Object	Role
Quantization group	Atomic policy unit with bit-width, parameter count, component label, and quantizability flag.
Raw INT4 budget	Feasibility constraint equal to assigning every quantizable group to INT4.
Sensitivity score	KL-based estimate of how much precision changes perturb target-task next-token distributions.
Greedy path state	Unique budget-feasible assignment produced by one accepted promotion plus any required demotions.
Coarse/refine candidate	Real path snapshot selected by realized INT8 mass, not by requested percentage.

Baselines and metrics. The final results should compare the selected mixed policy against BF16, uniform INT8, and uniform INT4 baselines under the same task protocol. For classification-style and math tasks, the table should report accuracy, correct count, token-limit hit count, average completion tokens, median completion tokens, p95 completion tokens, average latency, median latency, and p95 latency. For code-generation tasks, the table should report pass@1, execution status, capped-generation counts, and completion-

token statistics.

Serving caveat. Latency should be interpreted carefully because the current artifact path uses one-shot post-training quantization artifacts rather than an optimized mixed-precision serving stack. Accuracy and token usage are therefore stronger signals than raw latency at this stage.

6 RESULTS

This section is intentionally left as a placeholder for the final report numbers. It should not be replaced by informal run notes; it should contain final, reproducible tables and figures tied to exact artifact paths, benchmark splits, decoding settings, and policy identifiers.

The final version should also include an ablation table for the design choices that are central to the claim: task sensitivity profiling, exact-budget demotion repair, path-based coarse selection, and local refinement. If any benchmark shows no gain over uniform INT4, that failure case should be reported directly rather than hidden.

7 LIMITATIONS

The current method depends on the quality of the task sensitivity profiler. TAQ-KL-lite is a proxy: it measures output-distribution sensitivity under injected perturbations, not the exact downstream accuracy effect of assigning a group to a bit-width. This makes it useful for ranking, but not a proof that the best proxy group is always the best real promotion.

The search is structured rather than exhaustive. By constraining candidates to a greedy path, TA-MPQ gains interpretability and budget stability, but it can miss policies that require non-greedy combinations of promotions and demotions. The method is therefore best described as a practical exact-budget frontier search, not as a global optimizer over all INT2/INT4/INT8 assignments.

The current benchmark sizes for coarse and refine selection are modest. This makes tie handling and disjoint question sets important, but it also means small accuracy differences should not be overclaimed. The final Results section should report uncertainty or at minimum discuss sampling sensitivity.

The current serving stack is not an optimized production mixed-precision runtime. Raw footprint equality controls one important variable, but it does not guarantee equal throughput, equal peak VRAM, or equal kernel efficiency. A production claim would require an optimized INT4/INT2/INT8 serving backend or controlled comparisons against a strong INT4 runtime.

The current quantization granularity is also difficult for real

serving systems. Per-block component assignments are useful for search because they expose meaningful structure inside each transformer block, but they can create many small precision boundaries across attention and MLP projections. Future work should evaluate coarser granularities, such as whole-block, attention-versus-MLP, or shared-pattern grouping, to test whether most of the task-aware benefit can be preserved with policies that are easier to serve efficiently.

8 CONCLUSION

This paper frames task-aware mixed-precision quantization as exact-budget policy construction. The key idea is to spend INT8 precision where a task-sensitive profiler predicts high value, pay for it with low-risk INT2 demotions, and search over real path snapshots rather than requested percentages. The draft now defines the problem, method, experimental setup, placeholder result structure, and limitations in a stricter paper format. The remaining step is to fill the Results section with final frozen artifacts and to align every empirical claim in the Abstract and Introduction with those numbers.

REFERENCES

- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. NeurIPS 2022. <https://arxiv.org/abs/2208.07339>.
- Elias Frantar, Saleh Ashkboos, Torsten Hoeftler, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR 2023. <https://arxiv.org/abs/2210.17323>.
- Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. ICML 2023. <https://proceedings.mlr.press/v202/xiao23c.html>.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. MLSys 2024. <https://mlsys.org/virtual/2024/poster/2653>.
- Amit LeVi, Raz Lapid, Rom Himelstein, Chaim Baskin, Ravid Shwartz Ziv, and Avi Mendelson. You Had One Job: Per-Task Quantization Using LLMs' Hidden Representations. arXiv 2026. <https://arxiv.org/abs/2511.06516>.
- Mark Chen et al. Evaluating Large Language Models

Table 2. Placeholder for final accuracy and token-usage comparison. Fill this table only after freezing official artifacts and baselines.

Model	Split	Metric	Score	Avg tokens	P95 tokens	Token-wall hits
BF16	TBD	accuracy/pass@1	TBD	TBD	TBD	TBD
Uniform INT8	TBD	accuracy/pass@1	TBD	TBD	TBD	TBD
Uniform INT4	TBD	accuracy/pass@1	TBD	TBD	TBD	TBD
TA-MPQ mixed	TBD	accuracy/pass@1	TBD	TBD	TBD	TBD

Trained on Code. arXiv 2021. <https://arxiv.org/abs/2107.03374>.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. NeurIPS 2023. <https://openreview.net/forum?id=lqv610Cu7>.

Terry Yue Zhuo et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. ICLR 2025. <https://openreview.net/forum?id=YrycTjllL0>.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring Mathematical Problem Solving With the MATH Dataset. NeurIPS Datasets and Benchmarks 2021.